Using decPOMDPcoms to Holistically Model and Program Nanodevices and Emergent Nanonetworks

Florian-Lennert Adrian Lau*
Universität zu Lübeck
Lübeck, Germany
lau@itm.uni-luebeck.de

Tanya Braun
Westfälische Wilhelms-Universität Münster
Münster, Germany
tanya.braun@uni-muenster.de

Ralf Möller
Universität zu Lübeck
Lübeck, Germany
moeller@uni-luebeck.de

Stefan Fischer
Universität zu Lübeck
Lübeck, Germany
fischer@itm.uni-luebeck.de

ABSTRACT

In many areas like medicine, there is a wide range of promising use cases for nanostructures and nanonetworks. While many machine models and suggestions for individual components have been proposed in the past, the field lacks an enveloping modeling framework. Researchers rarely specify a machine model for their proposed solutions, and as a result, partial solutions are often incompatible. With a large set of devices, a partially observable stochastic environment, and noisy observations, many components of such nanoscale systems can be modeled as a decentralized, partially observable, Markov decision process with special communication capabilities (DecPOMDPcom). We extend and specify an existing set of component-based definitions to allow for much more precise specifications and the generation of autonomous programs for nanobots.

CCS CONCEPTS

• **Networks** → **Network architectures**; **Network components**; • **Hardware** → **Biology-related information processing**; • **Applied computing** → **Life and medical sciences**; **Computational biology**; *Consumer health*; • **Theory of computation** → **Machine learning theory**.

*Corresponding Author

NANOCOM '22, October 5–7, 2022, Barcelona, Spain

ACM ISBN 978-1-4503-9867-1/22/10.
https://doi.org/10.1145/3558583.3558814

KEYWORDS

molecular communication, nanonetworks, nano communication, nano, nanomedicine, POMDP, DecPOMDP, DecPOMDPcom

ACM Reference Format:
Florian-Lennert Adrian Lau, Tanya Braun, Ralf Möller, and Stefan Fischer. 2022. Using decPOMDPcoms to Holistically Model and Program Nanodevices and Emergent Nanonetworks. In *The Ninth Annual ACM International Conference on Nanoscale Computing and Communication (NANOCOM '22), October 5–7, 2022, Barcelona, Spain.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3558583.3558814

1 INTRODUCTION

Since Richard Feynman's revolutionary talk in the 1960s, many groundbreaking innovations in the field of nanotechnologies have been proposed [6]. Researchers from various domains contribute to insights concerning properties, ideas, and use cases concerning nanotechnologies. The interdisciplinary nature of the field makes it increasingly difficult to communicate, as all fields use a different set of terms to describe similar concepts [4]. The new field of nanonetworks combines insights from scientific disciplines like biology, chemistry, mathematics, medicine, and computer science. The main areas of nanonetworks are nano computation, nano communication, and nano construction. They result in the unique idea to create tiny devices or networks of such devices to perform tasks at the nanoscale. Tasks might be measuring environmental data or the manipulation of matter at the atomic level.

In the past years, many new building blocks and parts of nanodevices and nanonetworks have been proposed. A lot of publications agree upon a common set of components that are necessary for nanodevices to function. Components include actuators, sensors, antenna, information processors with memory, and a power supply E [1]. In [4] we made a first attempt at streamlining the vast amount of new research.

However, due to the heterogeneity of the field, we were forced to remain on a very general level.

In this paper, we extend the definitions from [4] and specify various proposed components in more detail. We leave most of the other factors intact, as they accurately describe the various environmental parameters of nanonetworks. However, we propose to specify the size requirements for nanodevice in regards to the different dimensions of the material or device [9]. As long as a device or material fulfills the range requirements of 1–4000 nm in one dimension, we consider it to be nanotechnology. Any device or material that meets this requirement is subject to the unique laws of nature that constitute the nanoscale.

Novel research in the area of nanonetworks suggests DNA-based nanonetworks as a pragmatic diagnostic tool [7]. It might even be possible to use nanodevices as a treatment method inside the human body [8]. As single devices might be too small to fulfill complex tasks, it is likely necessary for devices to collaborate. However, there are challenges due to the heterogeneity of nanoscale networks. In addition, each nanodevice possesses only local and incomplete information about the global system state. This is due to limited computational and memory resources as well as a local subjectivity to noise. All those limitations and the multitude of possible influence has to be taken into account when designing behavioral programs or *policies* for nanodevices. It is further necessary to assess the performance robustness or joint success rate of the networks as a tool for medical diagnosis.

We achieve this by modeling actuators, sensors, communication, information processing, locomotion, and the environment S using an extension of *partially observable Markov decision processes* (POMDP). The POMDP model has been introduced in the early phases of the development of autonomous robots. The model is suitable for autonomous devices and models most behavioral aspects well. It should be possible to express the core functionality of nanodevices and nanonetworks by extending the model by decentralized aspects and communication.

Therefore, this paper formalizes the setting as an offline decision-making problem, specifically decentralized, *partially observable Markov decision processes with communication* (DecPOMDPcom). The formalization fits well as a network contains a set of collaborating nanodevices, i.e.,*agents*, with limited capabilities in a *partially observable* environment whose interaction with the environment can be approximated with *stochastic* processes.

Thus, this paper aims to achieve several things: Researchers from many areas can more precisely and realistically describe the necessary components for their ideas to work. Further, it is easier to avoid implicit assumptions about the chosen machine model. And lastly, protocol or algorithm designers should have a much easier time modeling the combined behavior of nanonetworks.

The rest of the paper is structured as follows: Section 2 highlights the most important underlying definitions that this paper expands upon. Section 3 introduces and generalizes POMDPs by decentralized aspects and communication capabilities. The section thus suggests more specific models for many mandatory nanonetwork components. Section 4 implements a nanonetwork as a decPOMDPcom to illustrate the usefulness of the approach. Section 5 summarizes the paper and lists important open questions and future work.

2 RELATED WORK

The following definitions contain both necessary components as well as a set of constraints. The first category includes everything to physically enable the described behavior. Those components are *sensors* O, *actuators* A, components for *locomotion* L, as well as a component for *communication* C with other devices. Furthermore, there must be components that enable complex behavior. This includes a component for *information processing* I, optionally with *memory* M for data or programs, and a *timer* T. Last, nanodevices require a *power supply* E. In contrast to [4], the various device classes that follow are defined as a set of components instead of tuples and extended by more general nanoobjects [5].

Definition 2.1. A *nanoobject* $\mathcal{N}_O$ is an object of nanoscale size in at least one dimension in an environment S modeled as a random variable. A *nanoparticle* $\mathcal{N}_P$ is a nanoscale object of a maximum of 1–100 nanometers in all dimensions.

Examples are nanoparticles or grid-like sphere structures called clusters or Fullerene.

Definition 2.2. A *nanostructure* $\mathcal{N}_S = K_{\text{opt}}$ is an artificial nanoobject between 1–4000 nm in all dimensions. It is designed to perform a specific function in an environment S modeled as a random variable. A nanostructure consists of zero or more optional components $K_{\text{opt}} \subseteq \{\text{A,C,E,I,L,M,O,T}\}$.

After particles and nanoobjects, nanostructures are the most general, potentially complex structures that can be created for use at the nanoscale. The *nanoscale* corresponds to a size of one nanometer to a few micrometers in any dimension. Even though more than 1000 nanometers are technically no longer "nano-sized," the term has nevertheless been used in the literature because, for example, the human capillary system allows blood cells of four micrometers in size [9, 11].

Definition 2.3. A *nanodevice* $\mathcal{N}_D = K_{\text{mand}} \cup K_{\text{opt}}$ is a nanostructure that has a set of necessary components $K_{\text{mand}} = \{\text{E}\}$ and a set of zero or more optional components $K_{\text{opt}} \subseteq \{\text{A,C,I,L,M,O,T}\}$.

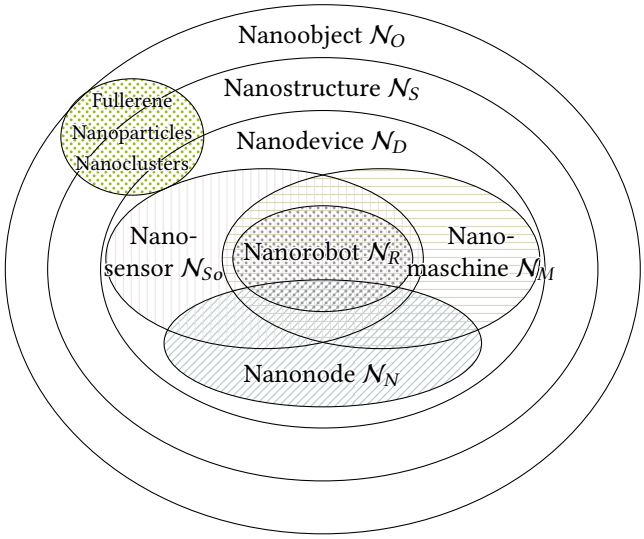

Figure 1: The relationship between various nanostructures and more specific nanodevices.

Nanodevices require a self-sufficient power supply E to distinguish them from fully passive nanostructures or particles. Just as with macroscopic machines, additional components may be present. A nanodevice is a nanostructure, and all other devices are defined on its basis. Figure 1 illustrates the hierarchical approach to the presented device definitions.

Nanodevices are expected to be used in unusual environments, such as the human body. The size of nanodevices poses new challenges such as molecular absorption in communication or quantum effects [3]. The existence of these requirements is illustrated by the S environment. The environment S designates a list of constraints or conditions. One possible constraint is the size of a device. For example, the human capillary system constrains some nanodevices to be less than four micrometers in size.

Definition 2.4. A *nanomachine* $\mathcal{N}_M = K_{\text{mand}} \cup K_{\text{opt}}$ is a nanodevice with necessary components $K_{\text{mand}} = \{\text{A,E}\}$ and a set of zero or more optional components $K_{\text{opt}} \subseteq \{\text{C,I,L,M,O,T}\}$, in an environment S.

This definition interprets nanomachines as very small machines. An artificial enzyme that folds proteins can be a nanomachine according to this definition, ignoring a direct energy supply. Similar to machines, the concept of a sensor node can be reformulated for the nanoscale [14].

Definition 2.5. A *nanosensor* $\mathcal{N}_{Se} = K_{\text{mand}} \cup K_{\text{opt}}$ is a nanodevice with the necessary components $K_{\text{mand}} = \{\text{E,O}\}$ and a set of zero or more optional components $K_{\text{opt}} \subseteq \{\text{A,C,I,L,M,T}\}$ in an environment S.

A simple nanomachine or nanosensor can endlessly repeat a simple task without ever evaluating information from the environment for interaction. However, to solve more complex tasks, it is necessary to autonomously create an abstraction of the environment S.

Definition 2.6. A *nanoagent* or *nano(ro)bot* $\mathcal{N}_R = K_{\text{mand}} \cup K_{\text{opt}}$ is an autonomous and programable nanodevice in an environment S. It has a set of mandatory components $K_{\text{mand}} = \{\text{A,E,I,M,O}\}$ and a number of zero or more optional components $K_{\text{opt}} \subseteq \{\text{C,L,T}\}$.

Since both computational space and memory are limited at the nanoscale, it can be assumed that many nanodevices must be involved in the solution of a problem in a network [1]. A prerequisite to participating in a nanonetwork is the ability of the potential participants to communicate. This allows nanodevices to exchange information so that a common goal can be worked towards.

Definition 2.7. A *nanonetwork* is a directed ad hoc graph $G = (V, E)$, where V is a set of *nanonodes* $\mathcal{N}_N$ operating in an environment S. Nanonodes are nanodevices that additionally have the necessary communication component $C \in K_{\text{mand}}$. E is a set of edges $e \in V \times V$.

Analogous to macroscopic networks, nanonetworks can be modeled as directed graphs. Here, vertices describe nanonodes, and edges correspond to the possibility of communication with other devices that are within range and not otherwise prevented from communicating. The environment S influences and constrains the graph significantly. Communication range and noise are conditioned by the environment. In an unstable or mobile environment, it is easy for both node positions and edges to be variable and thus harder to model.

In Section 3, the components O, A, I, L, and C will be directly modeled via decPOMDPcom.

3 DECPOMDPCOM FOR MODELING DEVICES AND NETWORKS

This section progressively defines a decision-making problem for modeling nanonetworks, going from single-agent / observable environment (MDP) to multi-agent / latent environment with communication (DecPOMDPcom). This section ends with an analysis of how to model the components mentioned above as a DecPOMDPcom.

3.1 Multi-bot Decision Making

MDPs are time-discrete stochastic processes. They model an agent's decision-making under circumstances where results are uncertain. MDPs are basically optimization problems. That is, solving them creates a program (policy) for the agent

by solving the underlying optimization problem. All the presented model variations are based on this principle. For a more detailed look into solution techniques, please refer to Oliehoek and Amato [10] as a starting point with a focus on DecPOMDPs. That said, generating behavioral programs for individual nanodevices is the objective of future publications.

The following definitions are based on [10, 12]. In the definitions, we use discreet random variables V, designated to as range, $ran(V) = \{v_1, \ldots, v_m\}$. If V is set to a value of its range $v \in ran(V)$, also called an *event*, we denote it as $V = v$ or v for short – if V is clear from its context. We call random variables with actions as ranges *decision random variables*. We denote sequences over a discrete time interval $[t_s, t_e]$ with subscript $t_s{:}t_e$, e.g., $(V_{t_s}, \ldots, V_{t_e}) = V_{t_s:t_e}$. We use $\circ$ to denote concatenation.

3.1.1 MDP. We model Markov decision processes as a sequential decision problem in a fully observable environment. An agent's preferences are time-independent, as implied by the Markov-1 transition model with additive rewards.

Definition 3.1. An (MDP) *model* $\mathcal{M}$ is a tuple (S, A, T, R),

- S, a random variable describing an agent's environment (the state space is the range of S),
- A, a decision random variable with a set of actions as range,
- $T(S', S, A) = P(S' \mid S, A)$, a *transition function*, with $T(S_0, ., .) = P(S_0)$ referring to an initial state prior, and
- $R(S, A)$, a *reward function*.

A so-called *policy* is a function $\pi : ran(S) \mapsto ran(A)$. The *semantics* of such a model is given by all possible policies, referred to as $\Pi_{\mathcal{M}}$.

Optional is a discount factor $\gamma \in [0, 1]$ (default 1). A policy is basically the behavioral program, which an individual agent follows. Future rewards are irrelevant when the discount factor is close to 0.

An *MDP* asks for the policy π^* that yields the maximum expected utility in $\mathcal{M}$. The Bellman equation can be used to describe the utility of a state $s \in ran(S)$ [2]:

$$U(s) = R(s) + \gamma \max_{a \in ran(A)} \sum_{s' \in ran(S)} T(s', s, a) U(s'). \quad (1)$$

The inner expression accumulates the *expected utilities*. The utility of state s' is multiplied with the probability of reaching s' from s applying an action a. The optimal policy π^* is:

$$\pi^*(s) = \arg\max_{a \in ran(A)} \sum_{s' \in ran(S)} T(s', s, a) U(s'),$$

i.e., the action a that yields the maximum expected utility. Based on Eq. (1), one can set up an iterative algorithm called value iteration that starts with random utilities per state and updates them given the current utilities of the other states until the differences between old and new utilities lies below an error margin ϵ. Russell and Norvig [12] serve as a starting point for further details and solution methods.

3.1.2 POMDP. A nanorobot has only limited information about its environment, which is not enough to deterministically determine the state of its environment. In decision making, such a setting is modeled as a POMDP, in which the state is considered latent, i.e., partially observable.

Definition 3.2. A (*POMDP*) model $\mathcal{M}$ is an MDP tuple extended with

- O, a set of possible observations as a range modeled by a random variable and
- a *sensor function* $\Omega(O, S, A) = P(O \mid S, A)$.

A POMDP still asks for the policy with the highest expected utility. However, now the state is latent, and additionally, observations and a sensor function are introduced that encode that an agent perceives observations about the environment as a form of evidence towards its estimation of the state it is in. It is possible to solve a POMDP by transforming it into a belief MDP. In a belief MDP, the latent S is replaced by a random variable B with an infinite set of belief states. A *belief state* $b \in ran(B)$ refers to a probability distribution over S. The state variables S, S' in T, R, and Ω are replaced by belief state variables B, B'. In the corresponding belief MDP, the optimal policy depends on belief states and not actual states. A belief MDP solution can be described by a conditional plan that combines `if. . . then . . . else` clauses. The `if` conditions depend on observations and the `if` and `else` blocks containing actions. Such a plan can be represented as a tree for each state, with the nodes labeled with actions and the edges labeled with observations. Each conditional plan p has a depth d and can be recursively constructed using plans of depth $d - 1$:

$$U_p(s) = R(s) + \gamma \sum_{s' \in ran(S)} P(s'|s, a) \sum_{o \in ran(O)} P(o|s') U_{p.o}(s') \quad (2)$$

where a designates the initial action in p and $p.o$ refers to the subplan of depth $d - 1$ for percept o that follows after a. Equation (2) yields a recursive algorithm that may stop given a predefined depth d or an error margin ϵ for the difference between plan utilities. All plans can be expressed as hyperplanes in the belief space. The plans with the highest values for some areas of the space are called dominating plans. Dominated plans have lower values over every possible belief state. Solving a belief MDP requires eliminating dominated plans, which that fewer conditional plans have to be considered for the recursive construction.

3.1.3 DecPOMDP. Unlike POMDP, a DecPOMDP models a set of collaborating agents with a common goal in a nanonetwork.

Definition 3.3. A *DecPOMDP* model $\mathcal{M}$ is a tuple $(\boldsymbol{I}, S, \boldsymbol{A}, T, R, \boldsymbol{O}, \Omega)$, with

- $\boldsymbol{I}$, a set of N agents,
- S, a random variable with a set of states as range,
- $\boldsymbol{A} = \{A_i\}_{i \in \boldsymbol{I}}$, a set of decision random variables A_i, each with a set of local actions as range, with $ran(\boldsymbol{A}) = \times_{i \in \boldsymbol{I}} ran(A_i)$ the set of *joint actions*,
- $T(S', S, \boldsymbol{A}) = P(S' \mid S, \boldsymbol{A})$, a transition function, with $T(S_0, ., .) = P(S_0)$ referring to an initial state prior,
- $R(S, \boldsymbol{A})$, a reward function,
- $\boldsymbol{O} = \{O_i\}_{i \in \boldsymbol{I}}$, a set of random variables O_i, each with a set of local observations as range, with $ran(\boldsymbol{O}) = \times_{i \in \boldsymbol{I}} ran(O_i)$ the set of *joint observations*, and
- $\Omega(\boldsymbol{O}, S) = P(\boldsymbol{O} \mid S)$, a sensor function.

A finite horizon τ, discount factor $\gamma \in [0, 1]$ (default 1), and margin $\epsilon > 0$ are optional. Each agent $i \in \boldsymbol{I}$ has a local policy $\pi_i : ran((O_{i,0:t})) \mapsto ran(A_i)$ mapping observation histories $o_{i,0:t}, t \leq \tau - 1$, to actions a, with $\boldsymbol{\pi} = (\pi_i)_{i \in \boldsymbol{I}}$ a *joint policy*.

Each agent has its own set of actions and possible observations in a DecPOMDP. Those may be identical for some agents, but the state and reward function are joint. Due to computational limitations, we assume that the joint state is not fully observable, which is typical for nanonetworks. If the joint state were observable, the DecPOMDP would simplify to a DecMDP. If both reward function and joint state are mostly independent per agent, the resulting POMDPs can be solved individually. With $N = 1$, the DecPOMDP also becomes a POMDP. The joint reward function ensures that the collaborating team receives rewards from the environment. The individual agent's reward functions $R_i(S, A_i)$ might conflict in a generalized observable stochastic game, which generalizes a DecPOMDP. The sensor function may also depend on a joint action, which does not change the problem in any major way [12].

Definition 3.3 uses observation histories of finite length as the domain of the local policies. Analogously to POMDPs, one could alternatively set up a belief state for each agent and then map belief states to actions in local policies. However, state estimation in DecPOMDPs per agent is a much harder task as each agent would also have to reason about the state that the other agents believe themselves to be in, which is why we opt for the definition as is. Solving a DecPOMDP looks for the optimal joint policy, i.e., with maximum expected utility, given a horizon τ,

$$MEU(\boldsymbol{\mathcal{M}}) = (\pi^*, U_{\mathcal{M}}(\pi^*)), \pi^* = \arg\max_{\pi \in \Pi_M} U_{\mathcal{M}}(\pi) \quad (3)$$

where $U_{\mathcal{M}}(\pi)$ is calculated recursively over τ steps and weighted according to the prior $T(S_0, ., .)$:

$$U_{\mathcal{M}}(\pi) = \sum_{s_0 \in ran(S)} T(s_0, ., .) U^{\pi}_{\mathcal{M}}(s_0, \emptyset_{0:0}) \quad (4)$$

$$U^{\boldsymbol{\pi}}_{\mathcal{M}}(s_t, \boldsymbol{o}_{0:t}) = R(s_t, \underbrace{\boldsymbol{\pi}(\boldsymbol{o}_{0:t})}_{=\boldsymbol{a}_t}) + \gamma^t \sum_{s_{t+1} \in ran(S)} T(s_{t+1}, s_t, \boldsymbol{\pi}(\boldsymbol{o}_{0:t})) \cdot \sum_{\boldsymbol{o}_{t+1} \in ran(\boldsymbol{O})} \Omega(\boldsymbol{o}_{t+1}, s_{t+1}) U^{\boldsymbol{\pi}}_{\mathcal{M}}(s_{t+1}, \boldsymbol{o}_{0:t+1}) \quad (5)$$

with $U^{\boldsymbol{\pi}}_{\mathcal{M}}(s_{\tau-1}, \boldsymbol{o}_{0:\tau-1}) = \gamma^{\tau-1} R(s_{\tau-1}, \boldsymbol{\pi}(\boldsymbol{o}_{0:\tau-1}))$ as the stopping point and $\boldsymbol{o}_{0:t+1} = \boldsymbol{o}_{0:t} \circ \boldsymbol{o}_{t+1}$.

3.1.4 DecPOMDPcom. Since nanorobots, in general, have a means of communication, the last extension to define in this paper regarding the sequential decision-making problem is that of a DecPOMDPcom, which extends a DecPOMDP with explicit means of communication. The messages in a DecPOMDPcom can be considered in three different dimensions, (i) as another action to perform by an agent, (ii) as another observation to perceive by an agent (emitted by fellow agents instead of the environment), and (iii) as a basis for communication cost. Because of this special standing of communication, we deal with it as an own component to be added to a DecPOMDP, even though a DecPOMDPcom (with explicit communication) can always be transformed into a DecPOMDP (with implicit communication).

Definition 3.4. A (*DecPOMDPcom*) model $\mathcal{M}$ is a DecPOMDP tuple extended with

- $\boldsymbol{C} = \{C\}_{i=1}^{N}$, being a set of random variables with identical atomic messages for communication including a null message $\perp_c$, with $ran(\boldsymbol{C}) = \times_{i=1}^{N} ran(C)$ the set of possible joint messages, and
- Γ, a cost function, which specifies the cost of transmitting an atomic message.

Communication is usually considered to be instantaneous and noise-free; dealing with other settings is part of future work. A message $c_i \in C$ refers to an atomic message sent by agent $i \in \boldsymbol{I}$ as a broadcast to all other agents. A null message indicates that an agent does not want to transmit any information. As defined above, each agent has the same set of messages available to send out, which is in contrast to how actions are handled where each agent may have its own set of actions available. Of course, one could also apply this idea to communication, with each agent having a random variable C_i with a range of locally specific messages.

Given messages, there is the option to include them in the functions of the DecPOMDP. A reasonable candidate is the reward function, resulting in $R(S, \boldsymbol{A}, \boldsymbol{C})$, to reward or punish certain communication apart from any cost the messages may. The transition function describes how the environment may change given its own inherent behavior and the agents'

actions. The agent's action may be influenced by a joint message, but given the action, the message and environment are independent of each other. The sensor function specifies the probability of perceiving an observation o out of the environment. This percept is also independent of any message. Therefore, incorporating a joint message here is less reasonable, although possible.

One needs to also specify the purpose of the messages, i.e., their semantics. One possibility is for each agent to share its local observations with all other agents to enable a joint state estimation. Another possibility is to communicate intent regarding the actions to perform. Depending on the semantics, solving a DecPOMDPcom may need to incorporate an update function where, depending on the information shared, actions might be chosen.

We now have all components necessary to model devices and networks by transferring their setup to DecPOMDPcom.

3.2 Mapping DecPOMDPcom to Components

This part transfers the device and network definitions to the DecPOMDPcom setting. Both the formalism and the nanoscale system have an environment in which they are represented by a random variable $S = \mathrm{S}$. We start with a component that does not have a direct counterpart in the model, namely, memory M. Then, we consider components with a direct connection like sensors O and actuators A. This includes locomotion L and communication C. We end with a discussion of power supply E, timer T, and information processing I, which get a reflection in the functions and solution of a DecPOMDPcom model.

The first component to consider is memory M. Memory is an important resource in any solving of an inference problem such as decision making. For any MDP-based problem, the model and problem themselves are considered to be specifiable given the available memory, as is each policy (locally for the agents and globally during calculation). A theoretical analysis regarding space requirements then may provide an upper bound on memory requirements for given solution approaches. There exists a solution approach to DecPOMDPs called memory-bounded dynamic programming that constrains its representation to available memory [13].

The next components to consider are the sensors O and actuators A. The sensors find a direct counterpart in the random variables O_i encoding the observations available to each agent i. These observations come from sensors in O in this nanoscale setting. During solving a DecPOMDPcom, all possible signal sequences are considered that O can emit up to a certain length τ. When acting out a policy, the signals that sensors from O generate (after possibly some preprocessing) become the observations for the agent upon which it decides on its next action. The sensor model Ω allows for modeling noise or accuracy regarding signal as a representation of a certain environment feature. Analogously, the actuators map to the decision random variables A_i and encode the actions available to each agent i. These actions need to be carried out by actuators in A. When acting out a policy and an agent has picked its next action given the observations from the sensors, the agent activates the actuator to which an action belongs. Locomotion is a special kind of action that, within the formalism of DecPOMDPcom, is treated as another possible action.

The last remaining component with a direct connection is communication C, for which we have introduced the communication variant of DecPOMDPs, namely, DecPOMDPcom. Without communication, a DecPOMDP covers all necessary components. The communication part is rather loosely defined in DecPOMDPcom and can therefore encode a multitude of communication to be broadcast between agents via acoustic, electromagnetic of molecular communication. The communication random variable C that is identical in this setup for all agents would then have a range of all possible messages that nanodevices could exchange. If no costs are associated with messages, the cost function Γ can be set to zero.

Next, we turn to the power supply E as a component without a direct counterpart at first glance. However, its effects can be reflected in the functions and policies of a DecPOMDPcom. Having a power supply implies a need for energy. The supply may get drained during acting and may therefore need to be stocked up. The recharging can be modeled as part of available actions and may actually yield a reward to encourage recharging at appropriate intervals.

A DecPOMDPcom is a formalization of a sequential decision problem, therefore, including a temporal dimension that is represented through discrete time steps t. In the transition function T, time finds its representation through the change from state s (at time point t) to state s' (at time point $t + 1$), which is encoded precisely in T. When acting out a policy, time elapses by having actions performed and observations made in each time step. When picking the next action from a policy based on an observation history, the history refers back to the last t time steps of observations. So a timer finds its way into the formalism of DecPOMDPcom.

The last remaining component is that of information processing I. It basically describes a program or acting routine for a device and therefore corresponds to a local policy π_i that is the solution to a DecPOMDPcom. Each agent in $\boldsymbol{I}$ gets one in the joint policy, which can be represented as a tree and basically qualifies as a look-up table, requiring no further processing power an agent during acting.

We specify an example of a DecPOMDPcom which corresponds to a nanonetwork in the next section.

4 EXAMPLE NANONETWORK

Based on the prior definitions, we can now model an example application as a decPOMDPcom. We do this by specifying and mapping all components of a DecPOMDPcom to the example nanonetwork from [8].

The different nanosensors and nanobots are modeled by the *set of agents* $\boldsymbol{I}$ with its K partitions. We consider κ types of nanosensors which respond up to κ different markers. There are possibly many copies of each type of nanosensor, and they form a partition $\Im_k^\kappa$ in $\boldsymbol{I}$. Nanobots are modeled similarly but react to message molecules. The ι sets of nanobots and up to ι corresponding messages form a partition $\Im_k^\iota$ in $\boldsymbol{I}$ each. Prior calculations suggest that $1.275 \cdot 10^{10}$ or more agents per partition might be necessary. This means that the set of agents is at least size $(\kappa + \iota) \cdot 1.275 \cdot 10^{10}$. Each type of sensor/bot has exactly one action and one observation.

Thus, there are two actions in each partition: (i) releasing tiles/medical payload, or (ii) inactivity, as well as two observations: (i) sensing markers or message molecules or (ii) sensing or receiving nothing.

The physical state of the system is determined by the presence of types of markers or messages. While there are countless intermediate assembly products, we only consider fully assembled message molecules. Thus, there are at least $2^\kappa \cdot 2^\iota$ possible states, given the simplified scenario.

The transition model T has to represent the presence of markers, which could follow a Poisson distribution. The presence of messages is fully determined by the markers, and the assembly process likely follows a log-normal distribution. Due to the remaining complexity, further simplifications might be necessary. The reward function has to encode the presence of markers to release medication when necessary. The sensor Ω has to reflect the correctness probability of observations, given the environmental noise.

There are several known and likely some unknown sources of errors: Nanosensors and nanobots might detect markers or message molecules even though none / wrong types are present. Message molecules might further be incorrectly assembled and still detected. It is also possible that both nanodevices do not detect input values in the presence of markers or message molecules. Given the extreme number of devices and possible interactions, it is not yet possible to solve decPOMDPcom in the general case. However, it might be possible to efficiently generate (joint) policies by modeling groups of nanodevices as indistinguishable (lifting).

5 CONCLUSION

This paper presents DecPOMDPcom as a means to model nanonetworks. DecPOMDPcom realistically models most components and constraints of nanonetworks with high accuracy. It is possible to represent partially observable environments without having any global information about the network state. Further, agents (nanodevices) can collaborate by communicating with each other and by sharing a single reward function that models a task to be solved. All actions a bot can perform are of stochastical nature, as any action at the nanoscale is subject to potential errors. Unlike methods based on ANN, the generated policies are explainable and there should thus be few medical limitations.

It might be possible to modify existing lifting solutions for the given representation or create new solutions. And lastly, it might be possible to apply recent results in lifted online decision-making to the given representation. It would thus be possible to generate optimal/sufficiently good policies for large nanonetworks.

REFERENCES

[1] Ian F. Akyildiz, Fernando Brunetti, and Cristina Blázquez. 2008. Nanonetworks: A new communication paradigm. *Computer Networks* 52, 12 (2008), 2260–2279.

[2] Richard Bellman. 1957. *Dynamic Programming*. Princeton University Press.

[3] Stephen Bush, Janet Paluh, Guiseppe Piro, Vijay Rao, Venkatesha Prasad, and Andrew Eckford. 2015. Defining Communication at the Bottom. *IEEE Transactions on Molecular, Biological and Multi-Scale Communications* 1, 1 (March 2015), 90–96.

[4] Florian Büther, Florian Lau, Marc Stelzner, and Sebastian Ebers. 2017. A Formal Definition for Nanorobots and Nanonetworks. In *The 17th International Conference on Next Generation Wired/Wireless Advanced Networks and Systems + The 10th conference on Internet of Things and Smart Spaces (NEW2AN ruSMART 2017)*. Springer, St.Petersburg, Russia.

[5] Stefan Engel. 2010. *Synthetische Nanomaterialien – Begriffswirrwarr und Auswirkungen auf den Arbeitsschutz*. Beuth Verlag, Berlin.

[6] Richard Philips Feynman. 1960. There's plenty of room at the bottom. *Engineering and science* 23, 5 (1960), 22–36.

[7] Florian Lau, Regine Wendt, and Stefan Fischer. 2021. DNA-Based Molecular Communication as a Paradigm for Multi-Parameter Detection of Diseases. In *4th ACM International Conference on Nanoscale Computing and Communication 2017 (ACM NanoCom'17)*. ACM.

[8] Florian-Lennert Adrian Lau, Florian Büther, Regine Geyer, and Stefan Fischer. 2019. Computation of Decision Problems within Messages in DNA-tile-based Molecular Nanonetworks. *Nano Communication Networks* (2019).

[9] Shahram Mohrehkesh, Michele Weigle, and Sajal Das. 2017. *Energy Harvesting in Nanonetworks*. Springer International Publishing, 319–347.

[10] Frans A. Oliehoek and Christopher Amato. 2016. *A Concise Introduction to Decentralised POMDPs*. Springer.

[11] M. Pierobon and Ian Fuat Akyildiz. 2010. A physical end-to-end model for molecular communication in nanonetworks. *IEEE Journal on Selected Areas in Communications* 28, 4 (May 2010), 602–611.

[12] Stuart Russell and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. Pearson.

[13] Sven Seuken and Shlomo Zilberstein. 2007. Memory-bounded Dynamic Programming for DEC-POMDPs. In *IJCAI-07 Proceedings of the 20th International Joint Conference on Artificial Intelligence*. IJCAI Organization, 2009–2015.

[14] Ming Xie. 2003. *Fundamentals of Robotics: Linking Perception to Action*. World Scientific Pub.